

Project Milestones: Scalable Flight Delay Prediction via Distributed PySpark ML Pipeline

Nirupama Balasubramanian (nirupamabala@umass.edu)

Dhevdharsan Bhavani Satish Kumar (dbhavanisat@umass.edu)

Jehan Kotwal (jkotwal@umass.edu)

Atharva Purohit (aspurohit@umass.edu)

1. Introduction

This project develops an efficient machine learning (ML) model to predict domestic U.S. flight delays. It addresses the Systems for Data Science requirements by leveraging Apache Spark and PySpark to implement distributed computing across the entire pipeline. Processing over 1 million flight records, the work demonstrates Spark's scalability and system efficiency in handling computationally intensive data preprocessing, feature engineering, and model training tasks that are unfeasible in single-node environments. Additionally we benchmark both Pyspark and Pandas to illustrate the performance differences.

2. Dataset - www.kaggle.com/datasets/patrickzel/flight-delay-and-cancellation-dataset-2019-2023

3. Repository - www.github.com/Jehan1241/FlightDelayPredictor

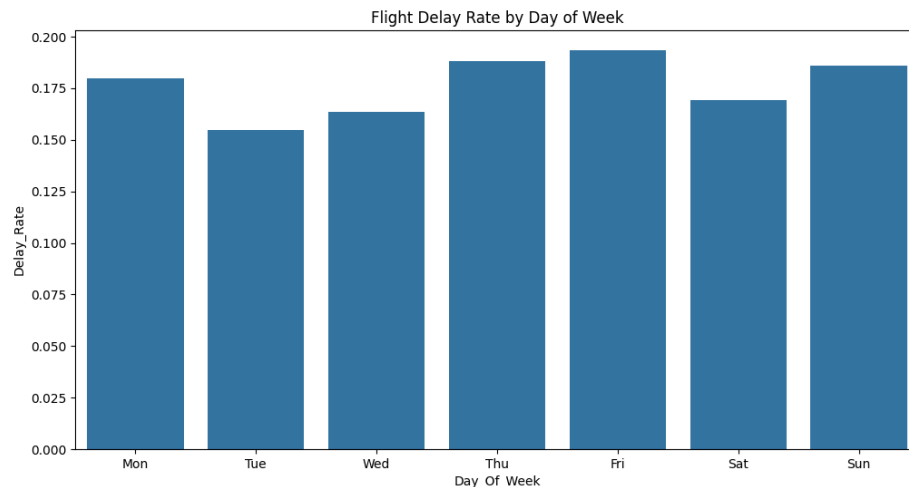
4. Tools

- Apache Spark (Core): Foundational distributed computing engine.
- PySpark (Python API): Primary interface for all data manipulation and ML logic.
- Spark DataFrame: Distributed data structure handling the 1M+ records.
- PySpark MLlib: Scalable ML library for algorithms and the Pipeline framework.
- Jupyter Notebook/VS Code: Interactive development environment.
- CSV/Parquet: Input/output data formats.
- Pandas: Alternative data processing engine to benchmark against.
- Git: Version control and code sharing.
- Matplotlib and Seaborn: To visualize data as graphs.

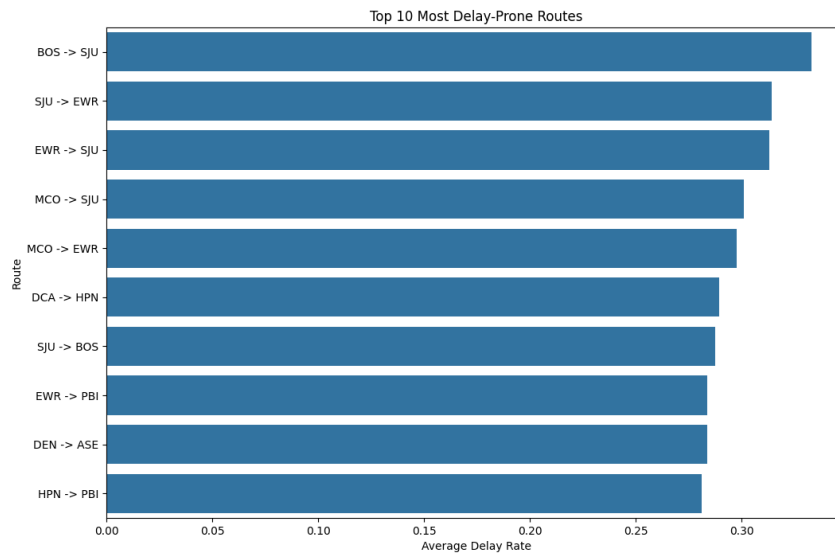
5. Milestone Goals (All Completed)

1. **Environment Setup:** We've created a git repo accessible to all team members.
2. **Data Ingestion:** The dataset was loaded into a Spark DataFrame and its integrity was verified.
3. **Initial Cleaning and Target Creation:** We cleaned the dataset by -
 1. Dropping columns which are not needed. Our final columns are - FL_DATE, AIRLINE, ORIGIN, DEST, CRS_DEP_TIME, CRS_ARR_TIME, CRS_ELAPSED_TIME, DISTANCE, ARR_DELAY
 2. Ensure valid departure and arrival times by filtering out impossible or broken values.
 3. Dropping the row for flights that are cancelled or diverted.
 4. Dropping the row if the flight distance is not between 10 miles to 10000 miles.
 5. Dropping the row if the average flight speed is not between 50 and 700 mph.
 6. Dropping the row if the origin is the same as the destination.
 7. Dropping empty strings.
4. **Exploratory Data Analysis (EDA):** We used SparkSQL to understand the data and answer a few questions, we then plotted the data and saved the graphs as a png-

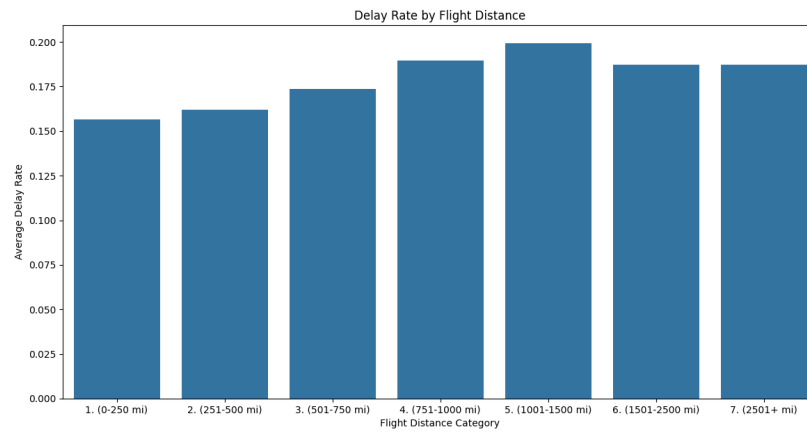
Q1 What's the delay rate by day of week?



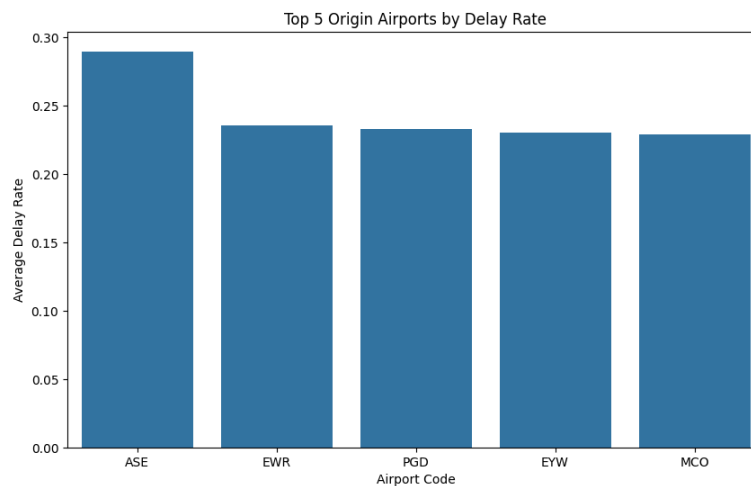
Q2 What are the 10 most delayed routes?



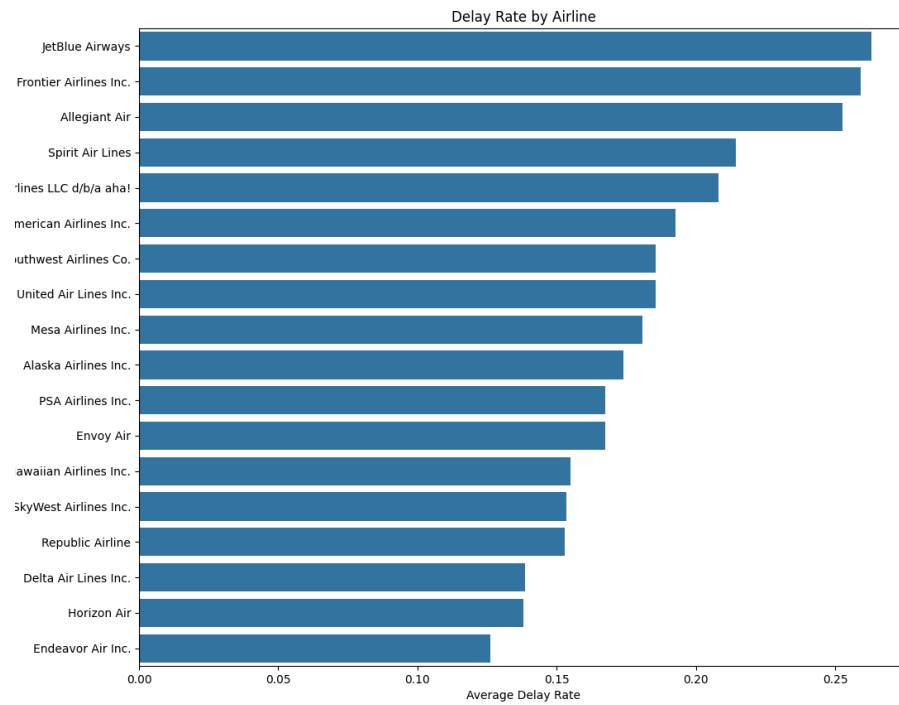
Q3 What's the flight delay rate by distance



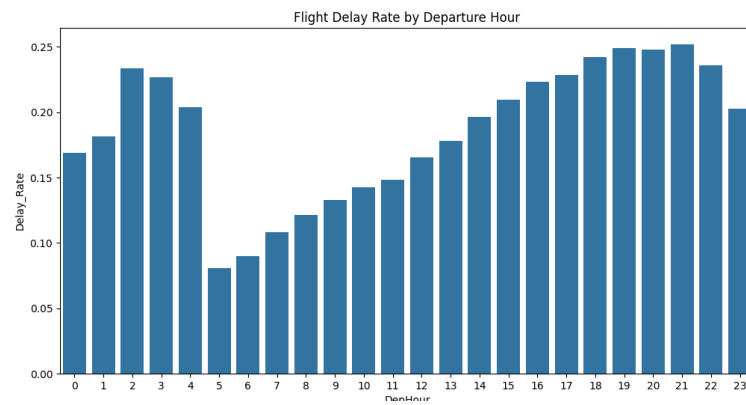
Q4 Which airports are most prone to delays?



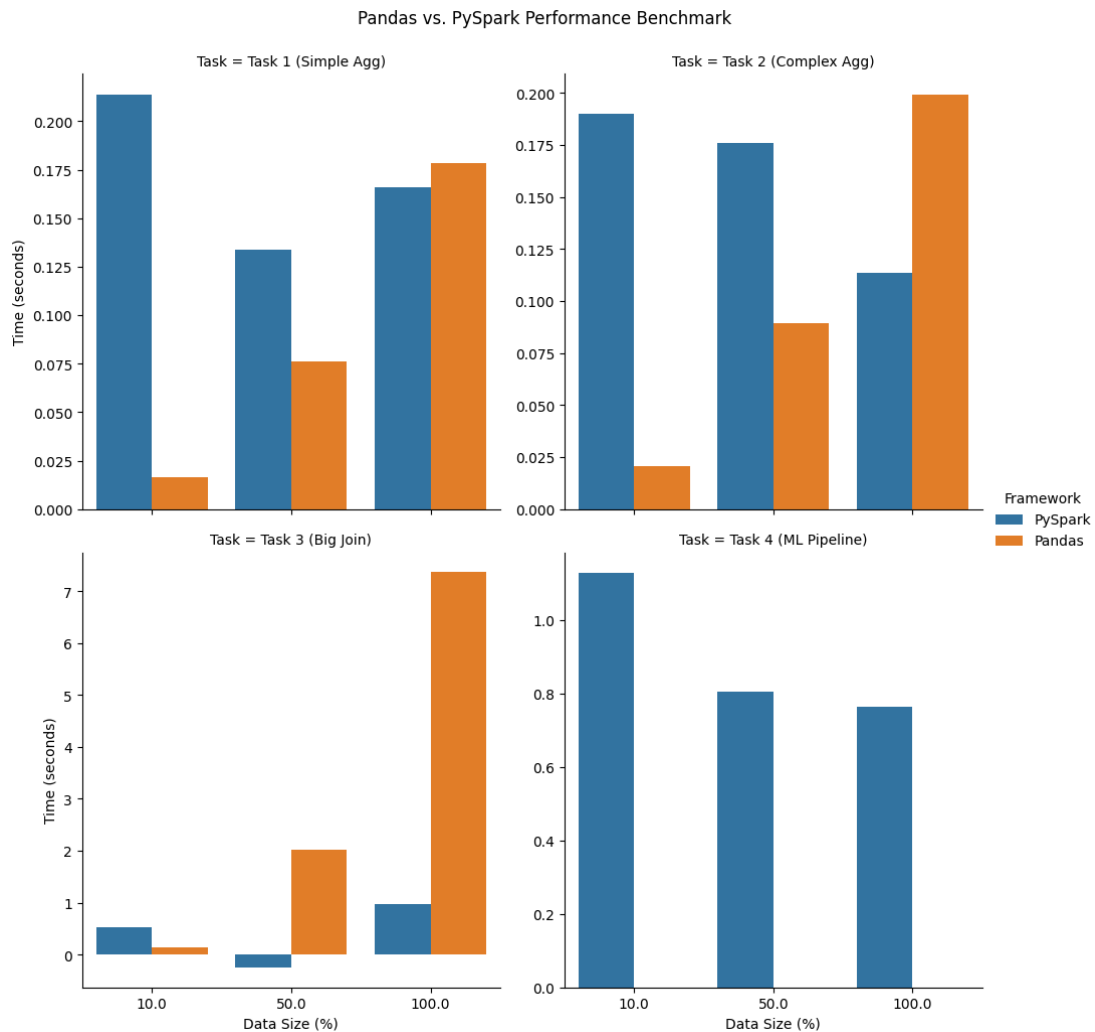
Q5 Which airlines have the worst delays?



Q6 How does the delay rate change by hour of day?



5. **Benchmarking:** We defined 4 tasks (Simple Aggregation, Complex Aggregation, A Heavy Join and an ML pipeline) and ran them in both Pandas and PySpark at 10% 50% and 100% of the dataset and compared the performance, we then plotted it.



6. **Feature Transformation:** We converted our categorical columns (AIRLINE, ORIGIN, DEST) into numerical indices via StringIndexer and converted those indices into binary vectors via OneHotEncoder.

6. Final Project Goals (To be completed)

1. **Pipeline Construction:** Use VectorAssembler to consolidate all features into a single vector column, then chain all steps into a single PySpark MLlib Pipeline for consistent, scalable transformation.
2. **Model:** A Logistic Regression Classifier is selected for its interpretability and effective performance in high-dimensional tasks.
3. **Distributed Training:** The complete pipeline is fitted to the training data, leveraging Spark's cluster resources for parallel, efficient training.
4. **Evaluation:** Performance metrics are calculated using the BinaryClassificationEvaluator.